

SCONE: A Logic Locking Technique Utilizing SMT Solver and Circuit Encoding Scheme for Efficient Hardware IP Protection

Zhaokun Han[†], Daniel Xing[‡], Kostas Amberiadis^{*}, Ankur Srivastava[‡], and Jeyavijayan (JV) Rajendran[†]

[†]Texas A&M University, [‡]University of Maryland, ^{*}National Institute of Standards and Technology (NIST)

[†]{hzhk0618, jeyavijayan}@tamu.edu, [‡]{dxing97, ankurs}@umd.edu, ^{*}kostas.amberiadis@nist.gov

Abstract—Multiple intellectual property (IP) protections have emerged to defeat security threats in integrated circuit (IC) supply chain. Among these, logic locking is regarded as a promising IP protection for its security. A state-of-the-art work uses stripped-functionality logic locking (SFL) technique with protected input patterns (PIPs) satisfying the distance of at least 2 (Dist2) property, or D2PIPs, for ensuring resilience against both input-output (I/O)-based and structural attacks. However, this approach has research challenges in scalability, flexibility, and security, as stated and discussed in our paper. Our paper solves these challenges by (i) utilizing a satisfiability modulo theories (SMT) solver and (ii) developing a secure circuit encoding scheme. SCONE, our secure logic locking technique, combines the two methods and meets all three challenges simultaneously. Our results show that SCONE improves scalability 350× on the IBEX processor (16K gates) and remains resilient against five I/O or structural attacks.

Index Terms—logic locking, encoding scheme, SMT solver.

I. INTRODUCTION

Due to the high cost of integrated circuit (IC) manufacturing, many IC design companies are now fabless and outsource manufacturing, packaging, and testing to third parties. As such, the IC supply chain is highly globalized, with many parties involved in the design and manufacturing process. However, outsourcing leaves ICs vulnerable to supply chain threats, such as intellectual property (IP) piracy, overproduction, counterfeiting, reverse engineering, and Trojan insertion [1]. Many IP protections against these threats have been proposed [2–4]. One promising approach is logic locking, which modifies a design with a key mechanism such that the locked design functions as designed only if given a correct key [2].

One of the most prominent logic locking techniques is stripped-functionality logic locking (SFL) [5]. SFL is provably secure against input-output (I/O)-based attacks [6] and practically secure against some structural attacks [7, 8] but vulnerable to others [9, 10]. While Han *et al.* [10] states that SFL-protected designs with protected input patterns (PIPs) satisfying the distance of at least 2 (Dist2) property, or D2PIPs, can gain structural resilience, such an approach is still impractical due to challenges in scalability, flexibility, and security.

The contributions of this work are as follows:

- We outline and discuss three challenges (i.e., scalability, flexibility, and security) in the state-of-the-art work [10].
- To solve the scalability challenge, we propose a novel approach using a satisfiability modulo theories (SMT) solver.
- The paper develops a novel secure circuit encoding scheme, enhancing both flexibility and security.
- By combining the two novel methods (using an SMT solver and developing a secure encoding), the proposed

logic locking technique, SCONE, solves all three challenges simultaneously and keeps resilient against all five I/O and structural attacks [6–10].

- Based on the designer’s capability and preference, this paper introduces two implementations of SCONE, including a discussion of their performance comparison.

In the remainder of the paper, Section II introduces the background, Section III introduces the proposed logic locking technique, SCONE, Section IV presents results, Section V provides the discussions, and Section VI concludes the contributions.

II. BACKGROUND

A. Potential Attacks on SFL

Potential attacks on SFL with D2PIPs [10] can be categorized into (i) I/O attacks and (ii) structural attacks.

I/O attacks. I/O attacks challenge the design’s IP by querying input patterns and observing correct outputs from an activated chip purchased from the market (a.k.a. oracle). The attacker determines the queries by analyzing the locked netlist. The most powerful I/O attack is the *Boolean satisfiability (SAT) attack* [6], as it breaks all logic locking techniques prior to the SAT attack. A SAT-capable attacker has a locked netlist and can query an oracle. The SAT attack eliminates incorrect keys iteratively by querying a distinguishing input pattern (DIP) till discovering the correct key.

Structural attacks. The locked netlist may contain clues to the correct key, which structural attacks exploit by analyzing the structure of the locked netlist. There are various structural attacks. The *AND-tree removal (ATR) attack* [7] identifies the AND-tree with a point function, which contains exactly one input pattern resulting in output 1, in the netlist and removes that tree to recover the functionality. The *signal probability skew (SPS) attack* [8] locates the wires with point functions by analyzing the probability of the signal to be 0/1 in the locked netlist and bypasses those wires to recover the functionality. The *function-analysis attack on logic locking (FALL)* [9] relies on the *unateness* property to locate key units and bypass them to retrieve the functionality. The *sparse prime implicant (SPI) attack* [10] analyzes the locked circuit’s prime implicant (PI) table¹ to infer the PIP information and unlock the design.

DISCLAIMER: The opinions, recommendations, findings, and conclusions in this article do not necessarily reflect the views or policies of NIST or the United States Government.

¹A PI table can be considered as a sum-of-products, where each PI represents a product. For all PI, all input bit values are from {0, 1, -}. The symbol “-” denotes a don’t-care bit, meaning its value does not affect whether the PI belongs to the ON-set/OFF-set.

This work is supported in part by NSF 1953285 and CSME 210205 grants.

B. Previous Approach: Using SFL with D2PIPs

Using SFL with D2PIPs [10] to protect the design provides provable security against both I/O and structural attacks.

An SFL-protected circuit has two main components: a stripped-functionality circuit (FSC) and a restore unit [5]. The FSC obfuscates protected input patterns (PIPs)—the FSC’s output differs from the original circuit’s output only when the input pattern is a PIP. With the correct key, the restore unit corrects the FSC’s flipped output. By XORing the FSC and the restore unit, the designer generates the SFL-protected circuit.

An attacker can break a SFL-protected design by finding the PIPs. However, for the SAT attacker, the probability of finding a PIP is exponentially small to each PIP’s length. Thus, SFL is provably secure against the SAT attack [5]. There are also multiple variants of SFL [5, 11–13], each targeting specific objectives such as better security, lower overhead, or a higher error rate. However, all these SFL techniques are broken by one of the aforementioned attacks, especially the SPI attack [10]. Only the state-of-the-art work [10] survives for its sufficient security against all aforementioned attacks.

SFL is practically secure against some structural attacks (i.e., ATR [7] and SPS [8]) but vulnerable to other structural attacks (i.e., FALL [9] and SPI [10]). As stated in Han *et al.* [10], for most randomly selected PIPs, the FSC’s netlist can leak PIP information, leading to structural vulnerabilities; thus, the defender needs to select specific PIPs to safeguard the design from structural information leakage. Han *et al.* [10] proposes that, from the PI table of the original circuit,² the defender can select PIs that keep at least distance 2 away from all other PIs in the PI table, as indicated in

$$\{D2PIP_{PI}\} = \{pi^* \mid \forall pi \neq pi^* \in F_{orig}^{ON}, Dist(pi^*, pi) \geq 2\}. \quad (1)$$

This property is referred to as the “Dist2 property,” and PIs meeting this criterion are “D2PIPs.” By choosing the D2PIPs as the PIPs, the FSC can conceal the PIPs information [10].

The previous work [10] protects the design with D2PIPs with the flow of (i) converting the netlist into a PI table for each output, (ii) analyzing each PI table to collect D2PIPs, and (iii) applying SFL with the collected D2PIPs to generate the SFL-protected circuit. The design protected by SFL with D2PIPs is secure against both I/O and structural attacks. However, research challenges exist in this approach, and we will demonstrate the challenges in Section III.

III. PROPOSED LOGIC LOCKING TECHNIQUE, SCONE

This section first outlines the challenges of scalability, flexibility, and security in the previous approach [10] (see Section III-A). To address all challenges, the paper introduces two methods: (i) utilizing an SMT solver to directly extract D2PIPs from the netlist, thereby achieving scalability (see Section III-B), and (ii) developing a circuit encoding scheme to introduce more secure PIPs, ensuring both flexibility and security (see Section III-C). Lastly, we propose a logic locking technique, *SCONE*, that safeguards the design against I/O and structural attacks while solving all three challenges simultaneously (see Section III-D).

²For simplicity, we focus on analyzing the ON-set of the original circuit, as well as the PIs and D2PIPs. A similar deduction can be made when considering the OFF-set case.

A. Research Challenges in Previous [10]

Scalability. To collect D2PIPs, the previous approach [10] involves converting the netlist into the PI table for each output. However, this circuit format conversion is a nondeterministic polynomial (NP) -hard process that scales exponentially to the circuit input size. For medium- and large-scale designs, this conversion is time-consuming and may not terminate within a reasonable time limit, resulting in a lack of scalability.

Flexibility. The high complexity of circuit format conversion often hinders the success of the PI table extraction within a reasonable time, and thereby, collected D2PIPs are rare. Moreover, for most practical designs, even assuming the success of every PI table extraction with an impractically huge time limit, D2PIPs are still rare.³ Consequently, the rareness of D2PIPs restricts the defender’s flexibility in selecting D2PIPs.

Security. The D2PIP’s length is often too small to protect the design from brute-force attacks due to the rareness of D2PIPs.

B. Fast D2PIPs Extraction Using an SMT Solver

We propose using the SMT solver to directly extract D2PIPs from the netlist, bypassing the process of generating a PI table.

According to Equation (1), for a D2PIP, all its distant-1 minterms⁴ in the Boolean space must be OFF-set. Otherwise, there exists a PI covering an ON-set minterm distant-1 away from the D2PIP, resulting in a contradiction. Thus,

$$\{D2PIP_{PI}\} = \{pi^* \in F_{orig}^{ON} \mid \forall m, Dist(m, pi^*) = 1 \Rightarrow f(m) = 0\}. \quad (2)$$

In Equation (2), pi^* is an ON-set D2PIP, “ $\forall m, Dist(m, pi^*) = 1$ ” denotes all minterm m ’s distant-1 to pi^* , and “ $\Rightarrow f(m) = 0$ ” represents the implies/implication of the OFF-set minterm m . However, Equation (2)’s condition still costs a high-execution effort for the SMT solver to search for D2PIPs. Thus, we narrow down to search for minterm D2PIPs only for the highest brute-force resilience, as opposed to PI D2PIPs with don’t cares and thereby less resilience. For minterm D2PIPs,

$$\{D2PIP_m\} = \{m^* \in F_{orig}^{ON} \mid \forall m, Dist(m^*, m) = 1 \Rightarrow f(m) = 0\}. \quad (3)$$

Using the SMT solver, we address the scalability challenge of searching for D2PIPs. Next, we demonstrate how the SMT solver searches for minterm D2PIPs based on Equation (3).

For a minterm D2PIP $m^* \in \{0, 1\}^n$ (n denotes the input size), we need the SMT solver to return the solution of each minterm D2PIP iteratively. For every minterm m distant-1 to m^* , there are n candidate m ’s, each flipping exactly one bit of m^* . Additionally, with the implies/implication “ $\Rightarrow f(m) = 0$ ”, the SMT solver applies the circuit’s Boolean logic of the output $f(\cdot)$ with ensuring that minterm m is OFF-set. In this way, the SMT solver detects a minterm D2PIP m^* . Iteratively, we collect other minterm D2PIPs. Theoretically, we can collect all minterm D2PIPs. However, if the circuit contains numerous minterm D2PIPs that cannot be fully collected within a reasonable time, we choose to gather partial minterm D2PIPs since we do not require all D2PIPs for protection.

³The rareness of D2PIPs is an inherent attribute for most practical designs. The designer usually applies cost-effective encoding (e.g., binary, Gray, and one-hot [14]) to produce compact and high-performance designs, which could result in this rareness [10].

⁴A minterm is a product where all input bit values are from $\{0, 1\}$ (no don’t care bit).

Although we can efficiently and successfully analyze all outputs by using a SMT solver, there are still very few minterm D2PIPs due to the rareness of D2PIPs and the restrictive condition of minterm D2PIPs. To address the challenges of flexibility and security, the next subsection proposes a novel method to introduce more secure PIPs with sufficient lengths.

C. Secure Circuit Encoding Scheme

We propose a circuit encoding scheme that (i) offers flexible choices of secure PIPs, and (ii) keeps resilient against I/O and structural attacks and gains security against brute-force attacks.

Assume the original circuit, C_{orig} , has n inputs ($in_1, in_2, \dots, in_n$). To introduce secure minterm PIP(s), the defender firstly selects ON-set minterm PIP(s) with the length of n . This selection provides the defender flexibility in choosing PIP(s). Assume that one selected PIP is $PIP^{before} = a_1 a_2 \dots a_n$, where $a_i \in \{0, 1\}$ and $i \in [1, n] \cap \mathbb{N}$, we need to expand inputs in the encoded circuit, $C_{encoded}$. To transform this selected PIP PIP^{before} into a PIP PIP^{after} with a longer length of $n + m$ ($m \in \mathbb{N}^+$), $C_{encoded}$ requires more input bits. Assume $C_{encoded}$'s inputs are ($in_1^{encoded}, in_2^{encoded}, \dots, in_{n+m}^{encoded}$), where $in_i^{encoded} = in_i$ for $i \in [1, n] \cap \mathbb{N}$. With m extra input bits, the Boolean space size increases exponentially, thereby enhancing brute-force resilience. Moreover, the choice of m provides the defender flexibility in determining the PIP size. When $i \in [n+1, n+m] \cap \mathbb{N}$, $in_i^{encoded}$'s encoding is determined by a random selection of ($in_{c_1^{i-n}}, in_{c_2^{i-n}}, \dots, in_{c_w^{i-n}}$), where $c_1^{i-n} \leq c_2^{i-n} \leq \dots \leq c_w^{i-n} \in \{1, 2, \dots, n\}$ and $w \in [2, n-1] \cap \mathbb{N}$. In this secure circuit encoding scheme, we define $in_i^{encoded} = in_{c_1^{i-n}} \oplus in_{c_2^{i-n}} \oplus \dots \oplus in_{c_w^{i-n}}$ when $i \in [n+1, n+m] \cap \mathbb{N}$. Therefore,

$$in_i^{encoded} = \begin{cases} in_i, & \text{if } i \in [1, n] \cap \mathbb{N} \\ in_{c_1^{i-n}} \oplus in_{c_2^{i-n}} \oplus \dots \oplus in_{c_w^{i-n}}, & \text{if } i \in [n+1, n+m] \cap \mathbb{N}. \end{cases} \quad (4)$$

After encoding, $PIP^{after} = b_1 b_2 \dots b_{n+m}$ satisfies

$$b_i = \begin{cases} a_i, & \text{if } i \in [1, n] \cap \mathbb{N} \\ a_{c_1^{i-n}} \oplus a_{c_2^{i-n}} \oplus \dots \oplus a_{c_w^{i-n}}, & \text{if } i \in [n+1, n+m] \cap \mathbb{N}. \end{cases} \quad (5)$$

Theoretically, there are total $\binom{n}{w}$ combinations for choosing ($c_1^j, c_2^j, \dots, c_w^j$); we set $w = \lfloor \frac{n}{2} \rfloor$ for providing the highest difficulty to the attacker by achieving the maximum of $\binom{n}{w}$. Moreover, to ensure that PIP^{after} satisfies the Dist2 property, we choose $m \geq 2$ for maintaining resilience against structural attacks. In the next subsection, we describe how we apply this encoding to protect the design by utilizing the SMT solver.

D. Logic Locking Technique, SCONE

We aim to protect the design while solving three challenges (i.e., scalability, flexibility, and security). This is accomplished by combining the approach of using the SMT solver for fast D2PIPs extraction with the secure encoding that introduces flexible and secure PIPs in the expanded Boolean space.

We now discuss the protection flow of SCONE. Firstly, we prefer to choose minterm D2PIPs if they are available. By utilizing the SMT solver, we can efficiently and effectively extract minterm D2PIPs. If no minterm D2PIP exists, we then apply the encoding to introduce flexible and secure PIPs and protect the design. A SCONE-protected design after encoding consists of three main components: an encoding

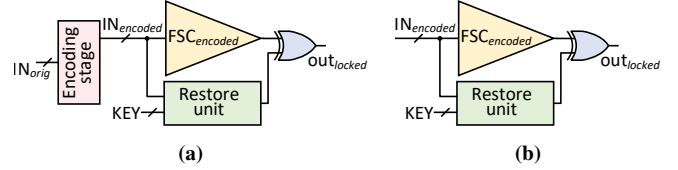


Fig. 1: The structure diagram of the SCONE-protected design in the implementation of (a) with ES in HW or (b) without ES in HW.

stage (ES), an FSC of $C_{encoded}$, and a restore unit. The relationship between inputs and outputs of the ES (IN_{orig} and $IN_{encoded}$) satisfies Equation (4). When with multiple PIP^{after} 's, $\{PIP_1^{after}, PIP_2^{after}, \dots, PIP_p^{after}\}$ ($p \geq 2$), the correct key is the concatenate, $key_c = \prod_{i=1}^p PIP_i^{after}$. We have two implementations based on whether the ES is implemented in hardware (HW):

Implementation with ES in HW (see Fig. 1(a)): If the designer does not apply the encoding during the design stage and maintains the original circuit's functionality, then the circuit keeps the original inputs IN_{orig} with a size of n . In this case, the defender needs to implement the configurable ES in HW to obfuscate the secure encoding. Notably, with the same input size of n , the resilience against brute-force attacks remains unchanged because the Boolean space keeps the same size of 2^n . In this implementation, the threat model allows the attacker to access (i) an oracle of the original design and (ii) the logic-locked netlist as shown in Fig. 1(a). The attacker's goal is to recover the original circuit's functionality.

Implementation without ES in HW (see Fig. 1(b)): The designer applies the secure encoding during the design stage, and the inputs of the produced design are $IN_{encoded}$, with a size of $n + m$. The attacker does not know how the defender determines the encoding as described in Equation (4). This implementation expands the input bits from n to $n + m$, thereby enhancing security against brute-force attacks (and other I/O attacks including the SAT attack) by significantly increasing the size of the Boolean space. Different from the previous threat model, in this implementation, the attacker is allowed to access (i) the oracle of the encoded circuit and (ii) the logic-locked netlist as shown in Fig. 1(b). The attacker aims to retrieve the encoded circuit's functionality.

IV. RESULTS

This section first discusses the experimental setup (Section IV-A). Then, we compare the distribution of D2PIPs and the execution time for extracting D2PIPs between the previous approach [10] and SCONE to highlight the flexibility of SCONE in Section IV-B. Next, we demonstrate the scalability by showing the execution time required by SCONE in Section IV-C. Following this, we conduct a security analysis with various attacks results on SCONE-protected circuits in Section IV-D. Finally, we present an overhead analysis in terms of power, performance, and area (PPA) in Section IV-E.

A. Experimental Setup

We perform experiments on a 32-core Intel Xeon processor at 2.6 GHz with 512 GB RAM. We use Berkeley ABC tool [15] for circuit format conversion, Synopsys Design Compiler [16]

for (re-)synthesis, and *Microsoft z3* as the SMT solver.⁵ Our test circuits consist of four ISCAS'85 circuits [17], nine ITC'99 circuits [18], and three other circuits (i.e., *Stanford MIPS* [19], *RISC-V IBEX* [20], and *Common Evaluation Platform (CEP) Global Positioning System (GPS)* [21]).

B. Collecting D2PIPs

Section III claims that (i) using the proposed approach utilizing the SMT solver accelerates extracting D2PIPs and (ii) using the circuit encoding scheme introduces flexible choices of minterm D2PIPs, compared to the previous approach [10]. Table I shows this comparison. Columns 3 & 6 present the success rate of extracting D2PIPs from the design, columns 4 & 7 reveal the maximum of D2PIP sizes, and columns 5 & 8 show the execution time to finish the D2PIPs extraction. Firstly, we can tell that the previous approach cannot ensure the success rate of D2PIPs extraction, while *SCONE* ensures the success rate of 100%. Next, *SCONE* is much more flexible than the one for the previous approach [10] for its enlarged maximum PIP size. “N.A.” denotes that the previous approach [10] fails to extract any D2PIP. Additionally, the maximum PIP size that *SCONE* can achieve is significantly higher than that of the previous approach [10], thereby enhancing security against brute-force attacks. Lastly, *SCONE* provides a much quicker analysis time than the previous approach [10]. “T.O.” denotes that the previous approach [10] cannot finish the D2PIPs extraction within 72 hours. Therefore, *SCONE* achieves more flexible PIP choices much faster while ensuring higher resilience against brute-force attacks, compared to the previous approach [10].

C. Execution Time of *SCONE*

Table II shows, for each circuit, the execution time required by *SCONE* when protecting the design. The chosen PIP sizes vary for different circuits. Here, we consider both implementations: one with ES in HW and one without ES in HW. From columns 4 & 5, we observe that the execution time required by the implementation without ES in HW is significantly less than that with ES in HW. Compared to the implementation without ES in HW, the implementation with

⁵We choose *Microsoft's z3* solver since it is a standard and widely-used SMT solver. Our theory is compatible with all SMT solvers, so other SMT solvers can also be used.

TABLE I: Comparison of D2PIPs extraction between the previous approach [10] and *SCONE*.

Circuit		Previous [10]			SCONE		
		Success rate (%)	Max. PIP size	Max. time (sec)	Success rate (%)	Max. PIP size	Max. time (sec)
ISCAS'85	c432	100	N.A.	431.6	100	72	1.9
	c1355	0	N.A.	T.O.	100	82	4.3
	c1908	16.0	N.A.	T.O.	100	66	3.1
	c7552	68.2	4	T.O.	100	388	45.8
		100	9	0.4	100	52	2.2
ITC'99	b12	100	26	0.4	100	74	1.7
	b13	100	11	0.1	100	46	0.4
	b14	53.5	29	T.O.	100	436	89.7
	b15	75.9	7	T.O.	100	612	281.7
	b17	60.4	6	T.O.	100	678	746.6
	b20	40.0	29	T.O.	100	570	305.3
	b21	35.2	29	T.O.	100	576	315.4
	b22	34.8	29	T.O.	100	578	322.2
Others	MIPS	80.6	3	T.O.	100	310	46.7
	IBEX	48.0	39	T.O.	100	902	748.2
	GPS	97.9	20	T.O.	100	240	102.9

TABLE II: The execution time of protecting the test circuit required by *SCONE*, including two different implementations.

Circuit	PIP size	Time (sec)	
		With ES in HW	Without ES in HW
ISCAS'85	c432	38	29.8
	c1355	43	36.3
	c1908	35	29.5
	c7552	126	314.3
ITC'99	b11	28	24.0
	b12	39	30.0
	b13	25	20.4
	b14	102	211.2
	b15	129	229.7
	b17	117	354.7
	b20	103	202.8
	b21	103	211.2
	b22	103	194.9
	MIPS	37	16.4
Others	IBEX	102	257.9
	GPS	103	226.3

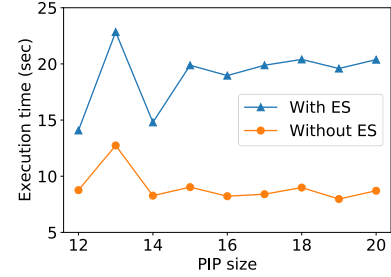


Fig. 2: The execution time of *SCONE* on b14 with different PIP sizes.

ES in HW requires additional execution time for building the configurable ES in HW, increasing the execution time. For the implementation with ES in HW, the defender first builds the configurable ES in a register-transfer level (RTL) Verilog file. Then, the defender uses *Synopsys Design Compiler* [16] to synthesize the RTL ES into its gate-level netlist. Finally, the defender combines the SFL-protected $C_{encoded}$ with the netlist of the configurable ES. As a result, building ES in HW requires significant extra time. Notably, even for large-scale designs such as IBEX and GPS, the execution time required by *SCONE* for the implementation without ES in HW is less than 80 seconds, demonstrating the scalability of *SCONE*.

We also include an analysis of the same design with varying PIP sizes. Fig. 2 shows how the protection execution time changes with different PIP sizes for the b14 design when using *SCONE*. From this figure, we observe that, for both implementations, the one without ES in HW always requires less time than the one with ES in HW. Additionally, varying the PIP size does not significantly affect the execution time for the implementation without ES in HW, indicating stable execution time with different PIP sizes. Thus, this result also supports the scalability of *SCONE*.

D. Security Analysis

In this subsection, we first have security analysis on *SCONE*; then, we showcase the vulnerability or infeasibility of existing SFL techniques with attack results.

Security Analysis on *SCONE*. To analyze the security of *SCONE*, we present Table III with all attack results (i.e., SAT [6], SPS [8], ATR [7], FALL [9], and SPI [10]) on the *SCONE*-protected circuits, where “T.O.” denotes that the SAT attack cannot terminate within 72 hours, and $\times$ indicates that

TABLE III: Attack results on *SCONE*-protected circuits.

Circuit		PIP size	With ES in HW					Without ES in HW				
			I/O SAT	Structural				I/O SAT	Structural			
				SPS	ATR	FALL	SPI		SPS	ATR	FALL	SPI
ISCAS'85	c432	38	T.O.	X	X	X	X	T.O.	X	X	X	X
	c1355	43	T.O.	X	X	X	X	T.O.	X	X	X	X
	c1908	35	T.O.	X	X	X	X	T.O.	X	X	X	X
	c7552	126	T.O.	X	X	X	X	T.O.	X	X	X	X
		28	T.O.	X	X	X	X	T.O.	X	X	X	X
ITC'99	b11	28	T.O.	X	X	X	X	T.O.	X	X	X	X
	b12	39	T.O.	X	X	X	X	T.O.	X	X	X	X
	b13	25	T.O.	X	X	X	X	T.O.	X	X	X	X
	b14	102	T.O.	X	X	X	X	T.O.	X	X	X	X
	b15	129	T.O.	X	X	X	X	T.O.	X	X	X	X
	b17	117	T.O.	X	X	X	X	T.O.	X	X	X	X
	b20	103	T.O.	X	X	X	X	T.O.	X	X	X	X
	b21	103	T.O.	X	X	X	X	T.O.	X	X	X	X
	b22	103	T.O.	X	X	X	X	T.O.	X	X	X	X
Others	MIPS	37	T.O.	X	X	X	X	T.O.	X	X	X	X
	IBEX	102	T.O.	X	X	X	X	T.O.	X	X	X	X
	GPS	103	T.O.	X	X	X	X	T.O.	X	X	X	X

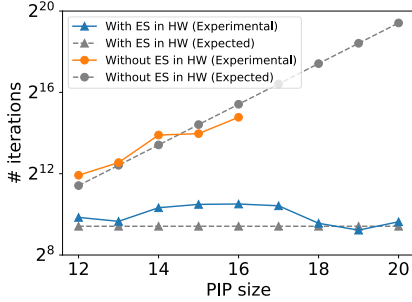


Fig. 3: The number of iterations required by the SAT attack [6] on b14 with different PIP sizes. The y-axis has an exponential scale.

the structural attack fails to break the design. All *SCONE*-protected circuits are resilient against any tested attack. Additionally, the PIP sizes chosen in Table III for *SCONE* are always greater than the maximum PIP sizes achievable by the previous approach [10]. For example, with GPS, the brute-force search space size is enlarged from 2^{39} (for the previous approach [10]) to 2^{103} (for *SCONE*). This indicates the much more enhanced resilience against brute-force attacks.

The SAT attack times out after 72 hours on all test circuits listed in Table III due to the large PIP sizes. To better understand the security of *SCONE* against the SAT attack, we present the SAT attack results on b14 with relatively small and varied PIP sizes in Fig. 3. The blue triangle marker denotes the experimental number of SAT iterations for the implementation with ES in HW, and the orange dot marker denotes the experimental number of iterations for the implementation without ES in HW. The grey triangle/dot marker denotes the expected number of iterations for the implementation with/without ES in HW. We can observe that, when changing the PIP size, the number of iterations is stable in a small range for the implementation with ES in HW, close to the expected number (the grey triangle markers in Fig. 3). This is because the SAT resilience is limited by the input size of the original circuit. Differently, when applying ES during the design stage (referring to the implementation without ES in HW), the number of iterations increases exponentially to the PIP size when the PIP size is in range of $[12, 16] \cap \mathbb{N}$, as shown in Fig. 3, where the y-axis is on a $\log_2$ scale. This tendency is close to the expected number of iterations (the grey dot markers in Fig. 3). When the PIP size is ≥ 17 , the SAT attack times out

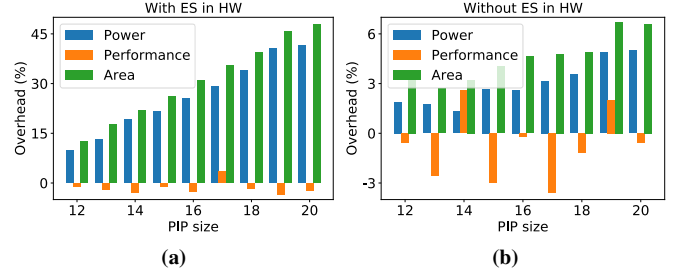


Fig. 4: PPA overhead analysis on b14 with different PIP sizes.

after 72 hours. Compared to the implementation with ES in HW, the one without ES in HW provides more I/O resilience and requires an exponentially large number of SAT iterations to the PIP size according to the curve's tendency.

Security Analysis on Existing SFL Techniques. We aim to showcase the insufficient resilience or infeasibility of the existing SFL techniques, including SFL-HD [5], SFL-flex [5], and the state-of-the-art approach [10]. Table IV presents the attack results on these existing SFL techniques. For both SFL-HD and SFL-flex, all locked circuits are vulnerable to the SPI attack [10]. For the state-of-the-art approach, using SFL with D2PIPs as described in Han *et al.* [10], the test circuit is either not able to be protected due to the lack of D2PIP, or the PIP size is too small to provide brute-force resilience thereby resulting in insufficient SAT resilience. Thus, not only the state-of-the-art approach [10] but also other SFL techniques [5], *SCONE* not only addresses the limitations in scalability, flexibility, and security, but also does not reduce resilience against various attacks [6–10]. Thus, *SCONE* outperforms the existing SFL techniques.

E. Overhead Analysis

Table V presents the PPA overhead results of two implementations. We can observe that without ES in HW incurs a lower overhead than the other one, particularly in power and area. Notably, for large-scale designs (e.g., IBEX and GPS), the PPA overheads are $< 10\%$ in area and power and with a negative performance overhead (improved performance). Thus, the implementation without ES in HW provides a more tolerable and acceptable PPA overhead.

To understand how the PPA overhead varies when changing the PIP size, we provide the PPA overhead analysis on b14 with different PIP sizes, as shown in Fig. 4. For both implementations, the power and area overheads go approximately linear to the PIP size. The negative performance overhead indicates the improved performance using *SCONE*, which is also common for other SFL techniques [5, 11]. Moreover, the implementation without ES in HW always has lower power and area overheads for avoiding the high cost in the configurable ES circuit.

V. DISCUSSION

This section discusses (i) the enhanced performance of *SCONE* in scalability, flexibility, and security compared to the previous approach [10], (ii) the comparison between two implementations, and (iii) prior related research works.

TABLE IV: Attack results on the existing SFL techniques, including SFL-HD [5], SFL-flex [5], and the state-of-the-art approach [10].

Circuit		SFL-HD [5]						SFL-flex [5]						Previous [10]					
		PIP size	I/O SAT	SPS	ATR	FALL	SPI	PIP size	I/O SAT	SPS	ATR	FALL	SPI	PIP size	I/O SAT	SPS	ATR	FALL	SPI
ISCAS'85	c432	36	311	✗	✗	✗	✓	36	115	✗	✗	✗	✓	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.
	c1355	41	T.O.	✗	✗	✗	✓	41	105	✗	✗	✗	✓	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.
	c1908	33	165	✗	✗	✗	✓	33	217	✗	✗	✗	✓	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.
	c7552	207	7,470	✗	✗	✓	✓	207	T.O.	✗	✗	✗	✓	4	12	✗	✗	✗	✗
ITC'99	b11	38	T.O.	✗	✗	✗	✓	38	62	✗	✗	✗	✓	9	434	✗	✗	✗	✗
	b12	126	T.O.	✗	✗	✗	✓	126	619	✗	✗	✗	✓	26	T.O.	✗	✗	✗	✗
	b13	63	T.O.	✗	✗	✗	✓	63	T.O.	✗	✗	✗	✓	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.
	b14	277	7,472	✗	✗	✗	✓	277	T.O.	✗	✗	✗	✓	17	T.O.	✗	✗	✗	✗
	b15	485	2,537	✗	✗	✗	✓	485	T.O.	✗	✗	✗	✓	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.
	b17	1,452	T.O.	✗	✗	✗	✓	1,452	T.O.	✗	✗	✗	✓	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.
	b20	522	2,281	✗	✗	✗	✓	522	T.O.	✗	✗	✗	✓	17	T.O.	✗	✗	✗	✗
	b21	522	T.O.	✗	✗	✗	✓	522	3,794	✗	✗	✗	✓	17	T.O.	✗	✗	✗	✗
Others	b22	767	T.O.	✗	✗	✗	✓	767	T.O.	✗	✗	✗	✓	17	T.O.	✗	✗	✗	✗
	MIPS	466	T.O.	✗	✗	✗	✓	466	350	✗	✗	✗	✓	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.
	IBEX	1,386	T.O.	✗	✗	✗	✓	1,386	T.O.	✗	✗	✗	✓	5	29	✗	✗	✗	✗
	GPS	10,009	T.O.	✗	✗	✗	✓	10,009	T.O.	✗	✗	✗	✓	20	T.O.	✗	✗	✗	✗

TABLE V: Overhead analysis for *SCONE*'s two implementations.

Circuit		# gates in C_{orig}	Overhead (%)					
			With ES in HW			Without ES in HW		
			Power	Performance	Area	Power	Performance	Area
ISCAS'85	c432	154	2624.7	11.6	2720.6	205.5	3.5	216.8
	c1355	191	882.2	13.9	1634.3	73.6	14.4	115.3
	c1908	218	814.0	-2.2	1213.7	81.0	-6.1	110.5
	c7552	1,145	1914.4	-7.7	2935.5	74.5	-2.8	89.5
ITC'99	b11	445	523.6	7.1	506.6	53.0	-1.8	51.8
	b12	986	572.6	-1.6	411.5	53.5	-2.2	33.1
	b13	253	864.9	8.0	691.3	99.0	-2.0	77.8
	b14	3,534	558.3	0.6	724.9	18.0	2.6	21.5
	b15	6,006	616.6	-4.5	578.1	16.2	-6.8	14.9
	b17	24,498	133.1	-4.2	130.5	3.6	-4.9	3.1
	b20	7,982	203.1	2.5	311.9	7.4	1.7	8.5
	b21	8,117	186.0	-2.1	302.6	7.0	-4.3	8.4
Others	b22	12,143	136.1	-1.5	204.7	3.3	-3.1	5.5
	MIPS	3,046	147.0	-8.4	119.2	11.6	-5.2	8.7
	IBEX	13,195	200.5	1.3	189.9	7.5	-4.2	7.1
	GPS	52,464	41.7	7.0	46.4	1.7	-3.3	2.3

A. Improvements in Three Perspectives

The experimental results and the related analysis indicate that *SCONE* overcomes the challenges in scalability, flexibility, and security. Table VI summarizes the comparison by listing the limitations in the previous approach [10] and the improvements that *SCONE* achieves. The summary table demonstrates the effectiveness of *SCONE* in overcoming three challenges.

TABLE VI: Comparison of the previous approach [10] and *SCONE*.

Perspective	Limitation in Previous [10]	Improvement in <i>SCONE</i>
Scalability	Exhausting search for D2PIPs and cannot guarantee success.	Fast D2PIPs extraction with a success rate of 100%.
Flexibility	D2PIPs are rare or do not exist.	Flexible choice of D2PIPs.
Security	Resilient against I/O and structural attacks but vulnerable to brute-force attacks.	Resilient against brute-force attacks while keeping resilient against I/O and structural attacks.

B. Two Implementations

There are two implementations of *SCONE* proposed in this paper, with and without ES in HW. Section IV demonstrates the strengths of the one without ES in HW from multiple perspectives: less execution time required by *SCONE*, higher resilience against SAT and brute-force attacks, and lower overheads (especially in power and area). Thus, if the designer is capable of applying the encoding during the design stage, we recommend choosing the implementation without ES in HW. The implementation without ES in HW (i) is more scalable in terms of execution time, (ii) provides higher security against I/O attacks, and (iii) holds a lower PPA overhead.

C. Prior Works on Secure Encoding Schemes

There are some prior and related works on secure encoding.

Active Metering [22] is the first work that proposes a secure encoding scheme for hardware metering. The protected design has additional flip-flops (FFs) and obfuscates the correct initial state. However, this technique is vulnerable [23]. *TriLock* [24] introduces a state encoding scheme for sequential logic locking for security purposes by encoding and decoding the inputs and outputs of FFs, and protects the design using the point-function method. One needs to be aware of the usage of the point function due to its potential structural vulnerabilities [7–10]. *STATION* [25] proposes *DisJoint* encoding, a prior secure encoding scheme for SFL. The *DisJoint*-encoded circuit has more D2PIP choices, providing the defender flexibility. However, *DisJoint* encoding targets FSM obfuscation only and requires the state-transition table (SST) of the original design. Converting a netlist to its SST is a time-consuming/exhausting process, so the scalability challenge exists in *STATION* as well.

D. Machine Learning-based Attacks

Other prior works have examined machine learning (ML)-based approaches to attack logic locking. They can be broadly categorized by their methods: those based on more conventional machine learning methods that work on fixed-dimension extracted features from locked circuits (e.g. *SAIL* [26], *Snap-Shot* [27], *SCOPE* [28]) and those that utilize graph neural networks which work directly on the graph structure of the locked netlist (e.g. *OMLA* [29], *INSIGHT* [30]). All of the previous works consider only key-gate insertion-based logic locking methods and do not challenge SFL techniques. Since *SCONE* protects the design's IP by relying on the mechanism of SFL, *SCONE* is resilient against existing ML-based attacks in logic locking.

VI. CONCLUSION

This paper proposes the secure logic technique, *SCONE*, addressing all challenges: scalability, flexibility, and security. Depending on the capability, the designer can choose the implementation either with or without ES in HW. We recommend the implementation without ES in HW for a faster protection process, higher security level, and a lower PPA overhead.

REFERENCES

- [1] M. Rostami, F. Koushanfar, and R. Karri, "A Primer on Hardware Security: Models, Methods, and Metrics," *Proceedings of the IEEE*, vol. 102, no. 8, pp. 1283–1295, 2014.
- [2] J. A. Roy, F. Koushanfar, and I. L. Markov, "EPIC: Ending Piracy of Integrated Circuits," *IEEE/ACM Design, Automation and Test in Europe*, pp. 1069–1074, 2008.
- [3] R. P. Cocchi, L. W. Chow, J. P. Baukus, and B. J. Wang, "Method and Apparatus for Camouflaging a Standard Cell based Integrated Circuit with Micro Circuits and Post Processing," *US Patent no. 20120139582*, 2012. [Online]. Available: <https://patents.google.com/patent/US20120139582/en>
- [4] F. Imeson, A. Emtenan, S. Garg, and M. Tripunitara, "Securing Computer Hardware Using 3D Integrated Circuit (IC) Technology and Split Manufacturing for Obfuscation," *USENIX Security Symposium*, pp. 495–510, 2013. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity13/technical-sessions/presentation/imeson>
- [5] M. Yasin, A. Sengupta, M. T. Nabeel, M. Ashraf, J. Rajendran, and O. Sinanoglu, "Provably-Secure Logic Locking: From Theory To Practice," *ACM SIGSAC Conference on Computer and Communications Security*, p. 1601–1618, 2017. [Online]. Available: <https://doi.org/10.1145/3133956.3133985>
- [6] P. Subramanyan, S. Ray, and S. Malik, "Evaluating the Security of Logic Encryption Algorithms," *IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, pp. 137–143, 2015.
- [7] M. Li, K. Shamsi, T. Meade, Z. Zhao, B. Yu, Y. Jin, and D. Z. Pan, "Provably Secure Camouflaging Strategy for IC Protection," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 8, pp. 1399–1412, 2019.
- [8] M. Yasin, B. Mazumdar, O. Sinanoglu, and J. Rajendran, "Security Analysis of Anti-SAT," *IEEE Asia and South Pacific Design Automation Conference (ASP-DAC)*, pp. 342–347, 2017.
- [9] D. Sirone and P. Subramanyan, "Functional Analysis Attacks on Logic Locking," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 2514–2527, 2020.
- [10] Z. Han, M. Yasin, and J. Rajendran, "Does logic locking work with EDA tools?" *USENIX Security Symposium*, pp. 1055–1072, 2021. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/han-zhaokun>
- [11] A. Sengupta, M. Nabeel, N. Limaye, M. Ashraf, and O. Sinanoglu, "Truly Stripping Functionality for Logic Locking: A Fault-Based Perspective," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 12, pp. 4439–4452, 2020.
- [12] B. Tan, R. Karri, N. Limaye, A. Sengupta, O. Sinanoglu, M. M. Rahman, S. Bhunia, D. Duvalsaint, R. D., Blanton, A. Rezaei, Y. Shen, H. Zhou, L. Li, A. Orailoglu, Z. Han, A. Benedetti, L. Brignone, M. Yasin, J. Rajendran, M. Zuzak, A. Srivastava, U. Guin, C. Karfa, K. Basu, V. V. Menon, M. French, P. Song, F. Stellari, G.-J. Nam, P. Gadfort, A. Althoff, J. Tostenrude, S. Fazzari, E. Breckenfeld, and K. Plaks, "Benchmarking at the Frontier of Hardware Security: Lessons from Logic Locking," *arXiv preprint arXiv:2006.06806*, 2020. [Online]. Available: <https://arxiv.org/abs/2006.06806>
- [13] Z. Han, M. Yasin, and J. Rajendran, "Multi-Objective Strategies for Stripped-Functionality Logic Locking," *IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–5, 2020.
- [14] W. Meeus, "Finite State Machine (FSM) encoding in VHDL: binary, one-hot, and others," <https://insights.sigasi.com/tech/vhdl-onehot-fsm/>, 2020, Last accessed on 09/08/2021.
- [15] R. Brayton and A. Mishchenko, "ABC: An Academic Industrial-Strength Verification Tool," *International Conference on Computer Aided Verification (CAV)*, pp. 24–40, 2010. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-14295-6_5
- [16] Synopsys, "Design Compiler NXT," <https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/design-compiler-nxt.html>, 2024, Last accessed on 04/02/2024.
- [17] M. C. Hansen, H. Yalcin, and J. P. Hayes, "Unveiling the ISCAS-85 Benchmarks: A Case Study in Reverse Engineering," *IEEE Design & Test of Computers*, vol. 16, no. 3, pp. 72–80, 1999.
- [18] S. Davidson, "Characteristics of the ITC'99 Benchmark Circuits," *IEEE International Test Synthesis Workshop*, p. 87, 1999. [Online]. Available: <https://www.cerc.utexas.edu/itc99-benchmarks/itsw.bench.pdf>
- [19] J. Hennessy, N. Jouppi, F. Baskett, and J. Gill, "MIPS: A VLSI Processor Architecture," *Springer VLSI Systems and Computations*, pp. 337–346, 1981. [Online]. Available: https://doi.org/10.1007/978-3-642-68402-9_37
- [20] L. Woods, Z. István, and G. Alonso, "Ibex: An Intelligent Storage Engine with Support for Advanced SQL Off-loading," *VLDB Endowment*, vol. 7, no. 11, p. 963–974, 2014. [Online]. Available: <https://doi.org/10.14778/2732967.2732972>
- [21] M. Hicks, P. Fiscarelli, and B. Chetwynd, "Common Evaluation Platform," <https://github.com/mit-ll/CEP>, 2021, Last accessed on 09/10/2024.
- [22] Y. M. Alkabani and F. Koushanfar, "Active Hardware Metering for Intellectual Property Protection and Security," *USENIX Security Symposium*, vol. 20, pp. 1–20, 2007. [Online]. Available: <https://www.usenix.org/conference/16th-usenix-security-symposium/active-hardware-metering-intellectual-property-protection>
- [23] M. Fyrbak, S. Wallat, J. Déchelotte, N. Albartus, S. Böcker, R. Tessier, and C. Paar, "On the Difficulty of FSM-based Hardware Obfuscation," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 293–330, 2018. [Online]. Available: <https://tches.iacr.org/index.php/TCHES/article/view/7277>
- [24] Y. Zhang, Y. Hu, P. Nuzzo, and P. A. Beerel, "TriLock: IC Protection with Tunable Corruptibility and Resilience to SAT and Removal Attacks," *IEEE Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 1329–1334, 2022.
- [25] Z. Han, A. Dixit, S. Patnaik, and J. Rajendran, "STATION: State Encoding-Based Attack-Resilient Sequential Obfuscation," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 10, pp. 2888–2901, 2024.
- [26] P. Chakraborty, J. Cruz, A. Alaql, and S. Bhunia, "SAIL: Analyzing structural artifacts of logic locking using machine learning," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 3828–3842, 2021, conference Name: IEEE Transactions on Information Forensics and Security. [Online]. Available: <https://ieeexplore.ieee.org/document/9478898/?arnumber=9478898>
- [27] D. Sisejkovic, F. Merchant, L. M. Reimann, H. Srivastava, A. Hallawa, and R. Leupers, "Challenging the Security of Logic Locking Schemes in the Era of Deep Learning: A Neuroevolutionary Approach," *J. Emerg. Technol. Comput. Syst.*, vol. 17, no. 3, pp. 30:1–30:26, 2021. [Online]. Available: <https://dl.acm.org/doi/10.1145/3431389>
- [28] A. Alaql, M. M. Rahman, and S. Bhunia, "SCOPE: Synthesis-based constant propagation attack on logic locking," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 29, no. 8, pp. 1529–1542, 2021. [Online]. Available: <https://ieeexplore.ieee.org/document/9466931/?arnumber=9466931>
- [29] L. Alrahis, S. Patnaik, M. Shafique, and O. Sinanoglu, "OMLA: An oracle-less machine learning-based attack on logic locking," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 69, no. 3, pp. 1602–1606, 2022. [Online]. Available: <https://ieeexplore.ieee.org/document/9539868/?arnumber=9539868>
- [30] L. L. Mankali, O. Sinanoglu, and S. Patnaik, "INSIGHT: Attacking Industry-Adopted Learning Resilient Logic Locking Techniques Using Explainable Graph Neural Network," *USENIX Security Symposium*, pp. 91–108, 2024. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/mankali>